CPU Scheduling Simulator

우진성

서론

CPU 스케줄러는 운영체제에서 여러 개의 실행 준비 상태에 있는, 즉 Ready 상태에 있는 프로세스들 중에서 어떤 프로세스에게 CPU를 할당할지 결정하는 시스템 소프트웨어이다. 즉, 어떤 프로세스에게 언제 어떠한 방식으로 CPU를 할당할지, 또 어떠한 방식으로 프로세스를 전환할건지 등 CPU의 작업을 스케줄링하는 전반적인 소프트웨어라고 말할 수 있다. CPU 스케줄러가 어떤 알고리즘을 통해 얼마나 효율적으로 프로세스에 CPU를 할당하느냐에 따라서 성능이 차이가 나게 되기에 운영체제에서 핵심이 되는 개념이라고 말할 수 있다.

본격적으로 직접 구현한 CPU Scheduling Simulator가 어떠한 특징을 가지고 있는지 설명하기 전에 프로세스의 상태에 대해서 짚고 넘어갈 필요가 있다. 서두에서 잠깐은 언급했지만 실행 준비(Ready) 상태에 있는 프로세스들 중에 특정한 기준을 토대로, 알고리즘을 토대로 CPU에 할당하는 것이다. 즉, Ready 상태에 있는 프로세스 중 특정한 어느 하나가 선택되어 CPU에 할당된다면 이는 Ready 상태에서 Running 상태로 바뀌게 된 것이다. Running 상태에 있는 프로세스 또한 다시 Ready 상태가 될 수 있으며, 혹은 작업이 완료될 수도, 혹으 I/O 요청으로 인해 waiting 상태로 바뀔 수 있다. 즉, 이러한 상태들의 전환이 발생할 때 프로세스들을 어떻게 처리할지가 CPU Scheduling Simulator의 핵심이라고 말할 수 있다.

이를 다시 말한다면 Context Switching을 처리하는 것이 본 Term Project의 핵심이다. Context Switching이란, CPU가 현재 실행 중인 프로세스 또는 스레드의 상태를 저장하고, 다음 실행할 프로세스나 스레드의 상태를 복원하는 과정이다. 이때 Context Switching이 발생하는 예시로는 I/O 요청, 자식노드 생성, time slice의 임계치 도달, preemption 발생 등 다양한 조건들이 있다. 이 중에서 본 프로젝트는 I/O 요청과 time slice의 임계치 도달, preemption 발생의 변수들을 다루었다.

즉, 본 프로젝트에서 다루는 프로세스들은 자식노드를 생성하지 않으며, I/O 요청이 존재하는 프로세스들을 스케줄링한다. 이러한 특징들을 가지고 있는 프로세스들을 처리하도록 직접 구현한 스케줄러의 전반적인 흐름은 다음과 같다.

1. **Ready 상태의 프로세스 관리**
2. **CPU에 할당 받을(Running 상태로 전환될) 프로세스 선정**
3. **Running 상태의 프로세스 처리**
4. **모든 프로세스들이 완료될 때까지 2로 돌아가 반복하여 작업 수행**

3번의 과정에서는 Context Switching이 발생한 경우 처리하는 것이 포함되어 있으며, I/O 작업 또한 CPU 스케줄러 알고리즘 내부에서 CPU와 별도의 흐름으로 처리되도록 구현되어 있다. 이제 흐름별로 구체적으로 스케줄러가 어떻게 동작하고, 어떻게 프로세스들을 처리하는지에 대해서 알아보도록 하자.

본론

1) 일반적인 CPU 스케줄링 시뮬레이터

직접 구현한 CPU 스케줄링 시뮬레이터를 살펴보기에 앞서 오픈소스로 구현되어져 있는 CPU 스케줄링 시뮬레이터에는 전반적인 공통점이 존재한다. 바로 I/O 요청이 발생하지 않는다는 전제 하에 알고리즘이 구현되어 있다는 것이다. 즉, context switching이 발생하는 조건으로, Round Robin 방식에서의 time slice expired 발생, Preemptive 기법을 도입한 알고리즘에서의 preemption 발생, 프로세스의 CPU burst time의 종료(즉, completed(terminated))를 고려하는 것이 대다수이다.

우선 프로그램 입력 예제이다.
왼쪽부터 차례대로 type / process_id / priority / computing_time을 의미한다.

0	1	25	80
0	2	15	40
0	3	8	30
1	0	0	0
0	4	12	10
0	5	-2	30
0	6	28	50
1	0	0	0
0	7	5	20
0	8	-3	40
0	9	13	60
1	0	0	0
0	10	24	45
-1			

프로그램 입력 예제

출처 : https://ggeunit.tistory.com/6

Input Process Data

Process 에 대한 입력 데이터

```
# Type ProcessID Priority Computing_Time
  0    1         25       80
  0    2         15       40
  0    3         8        30
  1    0         0        0
  0    4         12       10
  0    5         22       30
  0    6         28       50
  1    0         0        0
  0    7         5        20
  0    8         3        40
  0    9         13       60
  1    0         0        0
  0    10        24       45
  -1
```

- Type (0, 1)
  - 0 : Create Process
  - 1 : Time Quantum Exceeded
- ProcessID
  - ID of individual Process
- Priority
  - Priority of an individual Process
- Computing Time
  - Total amount of Compute Time required for an individual Process

출처 : https://github.com/KKodiac/CPUScheduling

위의 예시처럼 I/O 와 관련된 변수들은 배제한 채로 FCFS, Round Robin, Preemptive SJF, Preemptive Priority 등의 알고리즘을 구현하였다. 이에 반면 직접 구현한 프로세스들은 I/O 와 관련된 정보를 포함하고 있기에 이를 처리하는 별도의 장치들을 구혀한 것이 기존의 것과 차별화된 점이라고 말할 수 있다.

2) 직접 구현한 CPU 스케줄링 시뮬레이터

2-1) 프로세스의 생성

시뮬레이터에 사용될 프로세스를 생성하는 것이 출발점이다. 먼저 프로세스에 담긴 정보에 대해서 살펴보면, 크게 세 가지로 나눌 수 있다. 첫 번째로 프로세스 그 자체에 할당되는 고유한 정보, 두 번째로 스케줄링 알고리즘 간 성능을 비교하기 위한 정보, 마지막으로 현재 프로세스의 상태 정보 이렇게 구성되어 있다.

※ Process 구조체

대분류	변수 이름	변수 역할(정보)
프로세스 정보	pid	Process ID. 프로세스의 고유한 이름.
	arrival time	프로세스의 도착 시간.
	burst_time	프로세스의 CPU burst 시간. 즉 CPU에서 실제로 처리되는 시간.
	remaining_time	현재 프로세스에 남아있는 CPU 처리 시간. 초기값은 burst_time과 같으며, burst_time보다 항상 같거나 작음.
	priority	프로세스간 우선순위. 숫자가 작을수록 우선순위가 높음.
	io_count	I/O 요청 횟수.
	io_request_time	I/O 요청이 발생하는 시간. io_count만큼 존재하며, 1~(burst_time-1)의 시간에서 요청이 발생하도록 설정.
	io_burst_time	I/O를 처리하는 시간. io_request_time과 마찬가지로 io_count만큼 존재하며, io_request_time과 쌍을 이룬다.
	io_index	현재 처리할 예정인 I/O 요청을 뜻하는 변수.
성능을 비교하기 위한 정보	ready_time	ready 상태에 들어온 시각. 초기값은 arrival_time과 같음.
	finish time	최종적으로 프로세스가 완료되는 시각.
	waiting_time	프로세스가 끝나기까지 ready 상태에 있었던 총 시간의 합.
	turnaround_time	종료시각에서 도착 시각을 뺀 값. 즉, finish_time-arrival_time.
현재 프로세스 상태	in_ready	현재 프로세스가 ready 상태에 있으면 T, 그렇지 않으면 F.
	in_waiting	현재 프로세스가 waiting 상태에 있으면 T, 그렇지 않으면 F. 여기서 waiting 상태란 I/O 요청에 의해 I/O를 처리하는 과정에 있는 동안의 상태를 일컫음.
	in_running	현재 프로세스가 running 상태에 있으면 T, 그렇지 않으면 F.
	completed	프로세스가 완료되었다면(remaining_time == 0), T, 그렇지 않으면 F.

이러한 변수들을 가지는 구조체를 Process라 명명하여 다양한 알고리즘들을 활용하여 CPU 스케줄러를 구현하게 된다. 위의 표에서 알 수 있듯이 “프로세스 정보”들에서는 그 프로세스 자체가 가지고 있는 값들로 이 값들을 바탕으로 우선순위를 할당하며, preemption을 발생시킬 것인지, I/O 요청을 어떻게 처리할 것인지 결정한다. “성능을 비교하기 위한 정보”로서는 주로 시간에 관련된 변수들이다. CPU 스케줄링 알고리즘에 의해 처리되어진 프로세스들의 waiting time과 turnaround time을 통해 평균을 내어 성능을 비교하는 데 쓰인다. 이 중에서도 ready_time 변수의 경우에는 역할이 다른 time 변수들과는 다른데, 이는 특정 상태에서 ready 상태로 바뀌게 된 시각을 의미한다. 즉, CPU에 있다가 ready 상태로 들어온 시각, I/O를 처리하기 위해 waiting 상태에 있다가 ready 상태가 된 시각 따위를 저장하게 되는 변수이다. 마지막으로 “현재 프로세스 상태”는 프로세스가 어느 상태에 있는지 어떤 Queue에 존재하는지에 대한 정보를 제공한다.

그렇다면 이러한 프로세스들은 어떻게 생성할까? 직접 구현한 시뮬레이터에서는 크게 두 가지 방법으로 생성할 수 있다. 첫 번째로 사용자가 생성하는(manually) 방법, 두 번째로는 프로그램에서 임의로 생성하는(automatically) 방법이다.

첫 번째로 사용자가 생성하는 방법이다. 제일 먼저 생성해 낼 프로세스의 개수를 입력받는다. 이 후 각 프로세스당 차례로 pid, arrival_time, burst_time, priority를 입력받고, io_count, io_request_time, io_burst_time을 입력받는다. 이때 io_request_time을 발생하는 시간이 빠른 순으로 정렬하는 것으로 하나의 프로세스 생성을 마치게 된다.

```
Enter number of processes: 3

Enter Process ID: 1
Enter Arrival Time: 2
Enter Burst Time: 7
Enter Priority (1 is highest): 2
Enter number of I/O requests (0 to 6): 1
Enter I/O request time 1: 4
Enter I/O burst time 1: 2

Enter Process ID: 2
Enter Arrival Time: 1
Enter Burst Time: 3
Enter Priority (1 is highest): 3
Enter number of I/O requests (0 to 2): 0

Enter Process ID: 3
Enter Arrival Time: 4
Enter Burst Time: 13
Enter Priority (1 is highest): 1
Enter number of I/O requests (0 to 12): 3
Enter I/O request time 1: 5
Enter I/O burst time 1: 1
Enter I/O request time 2: 2
Enter I/O burst time 2: 4
Enter I/O request time 3: 10
Enter I/O burst time 3: 3
```

위 이미지처럼 사용자가 직접 프로세스에 대한 정보들을 생성할 수 있으며, 대분류상 “성능을 비교하기 위한 정보”와 “현재 프로세스 상태”는 알맞은 초기값으로 자동으로 설정한다. 이렇게 입력한 프로세스들을 표로 정리한 것이 아래의 이미지다.

```
Process ID      Arrival Time    Burst Time      I/O Count   Priority
1               2               7               1           2
2               1               3               0           3
3               4               13              3           1
I/O Request Times and Burst Times:
Process 1: (4, 2)
Process 2:
Process 3: (2, 4) (5, 1) (10, 3)
```

두 번째 방법으로는 프로그램에서 임의로 생성해내는 방법이다. 임의로 생성하기 위해서는 랜덤화가 중요한데 이를 사용자 정의 함수인 randomization()을 활용하여 구현한다.

```
   void randomization(배열 arr, 정수 n)
1.     for(i = n-1 to 1)
2.             0 <= j <= i 인 무작위의 수 j 생성
3.             arr[i] 와 arr[j] 를 swap
4.     end(for문)
5. end(함수)
```

이는 피셔 예이츠 셔플을 구현한 함수로 유한한 수열을 무작위의 순열로 섞기 위한 알고리즘이다. 로널드 피셔와 프랭크 예이츠의 이름을 따서 명명되었다. 작동되는 원리로는 기존의 배열에서 임의의 인덱스를 선택한다. 이는 위의 슈도코드의 2번 라인에 해당한다. 이렇게 선택된 임의의 값을 배열의 마지막 자리와 옮

기고, 마지막 인덱스를 제외한 값들에서 다시 무작위의 인덱스를 선택하게 된다. 즉, 임의로 선택된 순서대로 배열의 맨 뒤에서부터 재작성하게 되는 것이 피셔 예이스 셔플 알고리즘의 핵심이라고 말할 수 있다.
randomization 함수를 활용했을 때의 장점은 숫자를 무작위로 생성하되, 겹치지 않게 만들어 낼 수 있다는 점이다. arrival_time의 예시를 살펴보면,

1. 0부터 n 시각까지 순서대로 저장하는 배열(array[N+1]) 생성)
2. randomization(array, N+1)
3. 각각의 프로세스의 arrival_time 변수에 배열의 첫 번째 원소부터 차례로 할당

즉 1번 라인에서 (0, 1, 2, ... , N-1, N) 으로 차례대로 정렬되어 있는 배열을 2번 라인에서 randomization 함수를 활용하여 무작위로 (7, 5, 14, ... , 2, 3) 으로 재배치하게 된다. 최종적으로 원소들이 섞여져 있는 배열을 차례대로 실질적인 도착시간을 나타내는 Process 구조체의 arrival_time에 할당하게 된다. 즉, 첫 번째 프로세스의 도착시간은 7, 두 번째 프로세스의 도착시간은 5로 설정되는 것이다. 이러한 방식으로 priority 또한 구현할 수 있다. 이와 반면에 프로세스의 burst_time이나, io_count, io_burst_time의 경우 값이 중복되도 상관없기에 단순히 rand() 함수를 활용하여 구현한다.
여기서 주목해야할 부분은 io_reqeust_time의 무작위 처리 부분이다. io_request_time 변수의 경우, 제한된 범위 안에서, 제한된 조건 하에 변수를 알맞게 뽑아낸 뒤 이를 다시 정렬해야 한다. I/O 요청 시간의 경우 프로세스가 시작과 동시에, 종료하는 순간에는 CPU에서 처리되어야 하기에 시각이 0이나, 종료되는 시각에는 I/O 작업이 불가능하다. 즉 다시 말해, 1부터 cpu_burst_time-1 만큼의 범위 내에서 io_request_time이 생성되는 것이다. 여기서 그치는 것이 아니라 io_count만큼의 변수만 뽑아내야 하며, 이렇게 뽑힌 값들의 정렬을 보장해야 한다.

```
   void generate_sorted_io_request_times(배열 target, 정수 burst_time, 정수 io_count)
1.      1부터 burst_time-1 시각까지 순서대로 저장하는 배열(array[burst_time-1]) 생성)
2.      randomization(array, burst_time-1)
3.      for(i=0 to io_count-1)
4.              target[i] = array[i]
5.      end(for문)
6.      INSERTION_SORT(target)
7. end(함수)
```

여기 generate_sorted_io_request_times() 함수를 살펴보자. 1번 라인에서 I/O 요청 시간의 규칙인 1에서부터 (burst_time-1)만큼 순서대로 저장되어있는 배열을 생성한다. 이를 randomization() 함수를 통해 무작위화 한 후, target이라는 별도의 배열에 io_count만큼 저장하게 된다. 만약 burst_time이 10이고, io_count가 3이라면, 초기의 array는 (1, 2, 3, ... , 8, 9) 로 생성되며, randomization()을 거치면 무작위로 배치되어 (3, 7, 1, ... , 6, 2) 따위로 저장된다. 이를 다시 io_count인 3개만큼 뽑아내어 (3, 7, 1)만 배열 target에 저장하게 된다. 최종적으로 이를 정렬한 (1, 3, 7)이 배열 target의 최종적인 구조가 된다.
이렇게 “프로세스 정보”에 해당하는 변수들(arrival_time, burst_time, priority 등)을 랜덤하게 생성해낼 수 있다. 또한 각 변수들이 너무 퍼져있거나 너무 조밀하게 생성되는 것을 방지하고자 전역변수들을 설정하여 생성되는 범위를 조절한다.

```
#define PROCESS_COUNT 3
#define IO_COUNT 3
#define ARRIVAL_TIME 5
#define BURST_TIME 10
#define IO_BURST_TIME 5
#define TIME_QUANTUM 5
```

본 시뮬레이터에서는 위와 같은 전역 변수들을 설정하여 프로세스들을 제어한다. 여기서 한 가지 주목해야할 부분은 IO_COUNT와 관련된 부분이다. IO_COUNT의 경우, Process의 io_count를 결정하는데 사용되는 전역변수이다. 그러나 만약 임의로 생성된 프로세스의 (burst_time-1)보다 io_count가 더 큰 경우, io_request_time을 범위 내에서 생성할 수 없다. 다시 말해 [1, (burst_time-1)] 의 범위에서 io_request_time이 io_count 만큼 발생하는데, io_count가 burst_time-1 보다 큰 경우는 존재할 수 없는 것이다. 따라서 이 문제를 해결하고자 Process의 io_count가 생성될 수 있는 범위를 별도로 조정한다. 이와 과정을 모두 거쳐 "프로세스 정보"와 관련된 변수들에 무작위성을 부여하여 값을 설정할 수 있다.

```
Process ID      Arrival Time    Burst Time      I/O Count       Priority
1               0               8               2               3
2               3               2               0               2
3               5               16              1               1
I/O Request Times and Burst Times:
Process 1: (1, 1) (7, 1)
Process 2:
Process 3: (15, 2)
```

이는 전역변수들만 조정하여 프로세스들을 프로그램에서 생성한 결과이다. 이렇게 생성한 프로세스들을 어떠한 기준을 가지고 어떠한 방식으로 스케줄링을 하느냐에 따라 알고리즘이 달라지며 이 프로젝트에서는 FCFS, Nonpreemptive SJF, Nonpreemptive Priority, Round Robin, Preemptive SJF, Preemptive Priority 이렇게 6가지를 다룬다.

2-2) 스케줄링 알고리즘의 기본 구조

서론에서 언급했던 스케줄러의 전반적인 흐름을 다시 언급해본다면,

1. **Ready 상태의 프로세스 관리**
2. **CPU에 할당 받을(Running 상태로 전환될) 프로세스 선정**
3. **Running 상태의 프로세스 처리**
4. **모든 프로세스들이 완료될 때까지 2로 돌아가 반복하여 작업 수행**

모든 스케줄링의 알고리즘은 위와 같은 구조로 작동한다. Ready 상태의 프로세스를 관리하는 것부터 시작하는데, 여기서 "관리한다"라는 의미는 Ready Queue 내부의 프로세스들이 어느 위치에 존재하도록 할 것인지이다. 먼저 Ready 상태가 된 순서대로 배치할 것인지, 우선순위가 가장 높은 것부터 배치할지 기준을 정하여 정렬하게 된다. 즉, 무늬만 Queue 구조이지 기준을 가지고 정렬을 하는 구조기에 일종의 Max heap과 같은 구조라고 말할 수 있다.

이렇게 나름의 기준을 통해 관리된 Ready 상태에서 Running 상태로 바뀔 프로세스를 정하게 된다. 이

부분을 설명하기 전 preemption에 대한 개념을 먼저 짚어볼 필요가 있다. preemption은 선점, 우선권이라는 의미로 현재 CPU에서 처리 중이던 프로세스보다 우선순위가 높거나, Round Robin 방식에 의해 중단된 경우(time slice expired), 발생하게 된다. 처리 중이던 프로세스는 다시 Ready 상태로 전이되며, 그 자리를 새로운 프로세스가 채우게 된다. 이러한 preemption을 허용하는 방식을 preemptive, 그렇지 않은 경우를 nonpreemptive라 말하며 본 프로젝트에서는 SJF와 Priority에서 두 가지 방식의 알고리즘을 모두 구현한다. 즉 CPU에 할당 받을, Running 상태로 전환될 프로세스를 선정하는 것은 단순히 Running 상태가 휴지상태인 경우, Ready Queue에서 enqueue하는 것이 아닌 Ready queue의 front와 현재의 프로세스의 우선순위를 비교하는 과정을 내포하고 있는 것이다. 혹은 Round Robin에 의해 주어진 time slice에 도달한 경우, 자동적으로 Ready Queue의 front를 Running 상태로 놓게 된다. 이 부분에 대해서는 추후 Round Robin 알고리즘에서 상세히 다루도록 한다.

이렇게 선정된 프로세스를 이제 실질적으로 CPU에서 처리하게 된다. 프로세스가 CPU에서 처리되어지며, 남은 시간이 없어 종료될 때까지, preemption이 발생하기 전까지, I/O 요청이나 time slice expired가 발생하기 전까지 계속 CPU에서 작업을 수행하게 된다. 만약 종료된다면 completed 상태로 만들며, 그렇지 않은 경우 상황에 맞게 Ready 상태 혹은 Waiting 상태로 전이시키며, 모든 프로세스가 completed 상태가 될 때까지 2번과 3번의 과정을 반복하게 된다.

프로세스가 완료될 때, 즉 terminated 상태로 될 때 Waiting Time과 Turnaround Time을 계산하게 된다. 한 번 더 개념에 대해서 언급하자면 waiting time의 경우 Ready 상태에 있었던 총 시간을 말한다. 즉, CPU가 작업을 수행하고 있는 Running 상태, I/O 작업을 처리하고 있는 Waiting 상태를 Turnaround Time에서 제외한 시간이다. Turnaround Time의 경우 프로세스가 최종적으로 종료된 시점, 즉 finish time에서 arrival time을 뺀 값이다. 이렇게 계산된 값을 바탕으로 전체 프로세스의 평균을 구하며, 이 지표를 이용하여 알고리즘들 간의 성능을 비교하는데 사용하게 된다.

2-2-1) Ready Queue 관리

본 프로젝트에서 Ready 상태를 다루기 위하여 Queue를 활용한다. 기본적으로 Queue 구조는 선입선출이다. 즉, Queue에 들어온 순서대로 Queue를 빠져나가게 된다. 따라서 이는 FCFS 알고리즘에 사용되기에 적합하다.

```
for(i=1 to n)
    if(i번째 프로세스의 ready_time이 현재 시간보다 빠름) &(프로세스가 현재 Completed,
      Running, Waiting, Ready 상태 그 어느 것에 속하지 않음)
        enqueue(ready_queue, i번째 프로세스)
        i번째 프로세스의 상태 == Ready 상태 //Process->in_ready = true;
    end(if문)
end(for문)
```

위의 슈도코드의 핵심은 현재 시각과 프로세스의 ready_time 변수를 비교하는 것에 있다. ready_time이란 ready 상태로 전환된 시간을 의미하는 변수로 스케줄링 알고리즘을 구현하기 위해 임의로 도입한 개념이다. 모든 프로세스의 ready_time은 arrival_time으로 초기화된다. 이후 Running 상태나 Waiting 상태에서 다시 Ready 상태로 바뀐 그때의 시각으로 업데이트하게 된다. 특정 프로세스의 ready_time 값이 현재 시각보다 빠른 경우 Ready Queue에 enqueue하고 in_ready==true로 만들게 된다. 다만 프로세스가 이미 Ready 상태에 있거나, 특정 상태에 속한 경우에는 Ready Queue에 enqueue할 수 없다. 즉, 프로그램 상에서 아무 상태에도 속하지 않은 상태(in_ready = false, in_waiting = false, in_running = false, completed = false인 상태)의 프로세스만을 대상으로 enqueue하게 된다.

위와 같은 슈도코드는 FCFS에 가장 적합하며 실제로 구현한 FCFS 알고리즘의 경우 위처럼 구현되어져 있다. 반면, SJF와 Priority 방법은 선입선출이 원칙인 Queue의 구조와 부합하지 않다. 특정한 기준을 따라 Ready 상태에 존재하는 프로세스들 간 우선순위가 정해지며 이 순서를 따라서 Runninng 상태로 전이되어야 한다. 즉, 단순히 먼저 온 순서대로 처리되어지는 것이 아니기에 Ready Queue를 정렬할 필요가 있다. 이에 본 프로젝트에서는 기존의 Queue 구조를 유지하되, shortest job이나 priority를 기준으로 정렬하는 사용자 정의 함수를 통해 스케줄러를 구현한다.

```
1. void sort_ready_queue_by_remaining_time(&ready_queue)
2.      int size = length(ready_queue) //ready_queue에 존재하는 프로세스의 개수
3.      if(size <= 1)
4.              return;
5.      end(if문)
6.      Process* temp[size]
7.      int index = 0
8.      while(!is_empty(ready_queue)
9.              temp[index++] = dequeue(q); //기존의 ready_queue 구조 그대로 temp에 옮김
10.     end(while문)
11.     for(i = 0 to size-1)
12.             for(j = i+1 to size)
13.                     if(temp[i]->remaining_time > temp[j]->remaining_time)
14.                             temp[i]와 temp[j]를 swap
15.                     end(if문)
16.             end(for문)
17.     end(for문)
18.     for(i = 0 to size)
19.             enqueue(ready_queue, temp[i])
20.     end(for문)
21. end(함수)
```

위의 슈도코드는 Ready Queue를 remaining time이 가장 작은 프로세스 순서대로 정렬시키는 사용자 정의 함수이다. 함수의 구조를 보면, 먼저 Ready Queue의 프로세스가 1개인 경우, 정렬을 할 필요가 없기에 그대로 빠져나온다. 만약 2개 이상인 경우, 임의의 Ready Queue인 temp를 생성하여 기존의 ready_queue의 프로세스들을 차례대로 temp에 옮긴다. 이 과정이 6번에서 10번 라인에 해당한다. 이후 각 프로세스들의 남은 burst time인 remaining time의 값을 비교하여, 값이 작은 순으로 정렬하게 된다. 최종적으로 이렇게 정렬된 함수를 다시 차례대로 기존의 Ready Queue에 enqueue하게 된다. 이 과정을 통하여 무늬는 Queue이지만 SJF 알고리즘을 구현하기 위해 실질적으로는 ordering이 조재하는 heap과 같은 구조를 띄게 하는 함수를 정의하게 된다. 이 구조와 비슷한 방식으로 Priority 알고리즘에도 적용하게 된다.

```
void sort_ready_queue_by_priority(&ready_queue)
    int size = length(ready_queue) //ready_queue에 존재하는 프로세스의 개수
    if(size <= 1)
        return;
    end(if문)
    Process* temp[size]
    int index = 0
    while(!is_empty(ready_queue)
        temp[index++] = dequeue(q); //기존의 ready_queue 구조 그대로 temp에 옮김
    end(while문)
    for(i = 0 to size-1)
        for(j = i+1 to size)
            if(temp[i]->priority > temp[j]->priority)
                temp[i]와 temp[j]를 swap
            end(if문)
        end(for문)
    end(for문)
    for(i = 0 to size)
        enqueue(ready_queue, temp[i])
    end(for문)
end(함수)
```

위의 슈도코드는 프로세스의 Priority 값에 따라 Ready Queue를 정렬하는 사용자 정의 함수이다. 앞서 살펴본 방식과 똑같은 구조를 가지고 있으나, remaining time의 부분이 priority로 대체되었다는 점이 유일한 차이점이다. Priority의 값이 작으면 작을수록 우선순위가 높다. 즉, Prirority 값을 오름차순으로 정렬하는 구조를 지니고 있는 것이다. 이 함수를 활용하여 Priority 알고리즘을 구현할 때 Ready Queue를 관리할 수 있게 된다.

2-2-2) Current Process 지정

2-2-1)의 함수들이나 알고리즘들을 통하여 관리된 Ready Queue를 활용하여 현재 Running 상태에서 작업할 프로세스를 지정하게 된다. 즉 current_process를 지정하여 다음 구조인 실질적으로 Running 상태에서 CPU 작업을 수행하게 되는 것이다. 앞서 Ready Queue를 관리하는 것은 FCFS 방식이냐, SJF, Priority냐에 따라서 적용하는 함수들이 달랐다면, current_process를 지정하는 것은 preemptive 방식이냐, nonpreemptive 방식이냐에 따라 차이점이 발생한다.

```
if(current_process == NULL && !is_empty(ready_queue))
    current_process = dequeue(ready_queue)
    current_process->in_running = true
    current_process->in_ready, in_waiting = false
end(if문)
```

위의 슈도코드는 preemptive와 nonpreemptive 방식 모두에 적용되는 공통적인 요소이다. 현재 current_process가 존재하지 않지만, ready_queue에는 프로세스가 존재할 때, ready_queue의 front

프로세스가 current_process가 된다. 즉, Running 상태의 프로세스가 존재하지 않을 때, Ready 상태의 프로세스를 Running 상태로 전환시키는 코드이다. 이렇게 전환된 프로세스의 상태를 Running으로 바꿔주기 위해 현재 프로세스 상태를 나타내는 변수들을 알맞게 업데이트하게 된다. 구조를 봐서 알겠지만 preemption은 배제되어 있는 상태인데, 이를 적용하는 preemptive SJF와 preemptive Priority 알고리즘에서는 어떠한 방식으로 current_process를 지정할 수 있을까?

```
6. if(ready_queue에 프로세스 존재 && current_process 존재 && current_process의 remaining
   time이 ready_queue의 front 프로세스보다 클 때)
7.        current_process->in_ready, in_waiting, in_running = false
8.        current_process = dequeue(ready_queue)
9.        current_process->in_running = true
10.       current_process->in_ready, in_waiting = false
11. end(if문)
```

위의 슈도코드를 앞선 1~5번 라인에 붙여서 preemptive SJF를 구현할 수 있다. 위의 코드의 구조를 살펴보면, currnet_process가 존재하고, ready_queue의 프로세스가 존재하는 경우에 작동한다. 즉, 비교대상이 있다는 말이며 이는 preemption이 발생할 조건을 만족한다는 의미와 같다. 여기서 SJF 스케줄링 알고리즘의 원리인 Shortest Job First, 즉, remaining time이 작은 프로세스일수록 우선순위라는 조건에 부합하는가를 확인하게 된다. Ready Queue는 이미 정렬되어 있는 상황이기에 front 프로세스가 가장 작은 remaining_time을 보유하고 있다. 이를 현재의 프로세스와 비교하였을 때, 현재의 프로세스의 remaining_time이 더 크다면, 즉 preeption이 발생해야 되는 상황이면, current_process의 현재 상태를 나타내는 변수들을 다 false로 만들어 프로그램 상에서 아무 것도 속하지 않는 상태로 만든다(이후 current_process는 특별한 이슈가 발생하지 않는다면 ready queue에 곧바로 들어갈 것이다). 이후 ready_queue의 front를 current_process로 업데이트하고, 이와 더불어 이 프로세스의 상태를 running으로 나타내어 준다. 만약 이 if문의 조건이 참이라면, preemption이 발생한 것이고, 그렇지 않다면 preemption이 발생하지 않은 것이다.

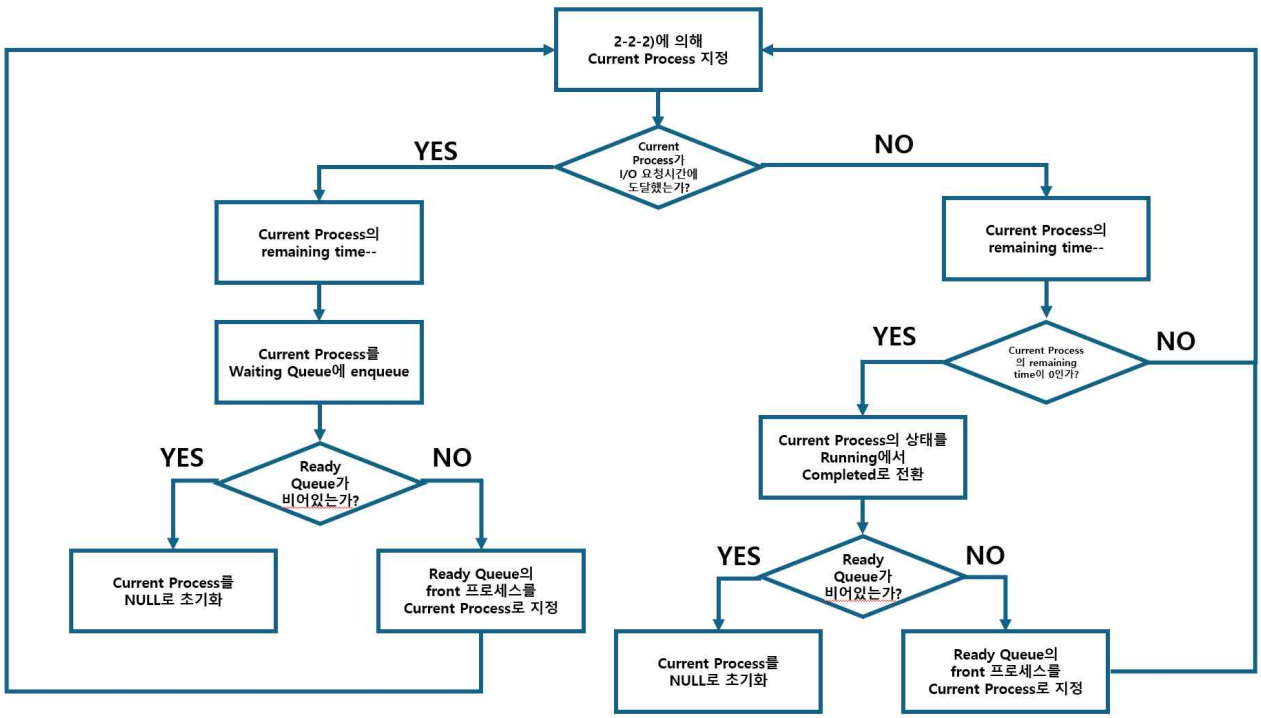
Preemptive Priority의 경우, 6~7번 라인의 “current_process의 remaining time이 ready_queue의 front 프로세스보다 클 때”를 “current_process의 Priority 값이 ready_queue의 front 프로세스보다 클 때”로 바꿔 적용할 수 있다. Priority의 값이 작으면 작을수록 우선순위가 높기에, current_process의 Priority 값이 ready_queue의 front 프로세스보다 크다는 것은 우선순위에서 밀린다는 뜻이기도 하기에 preemption이 발생한다.

2-2-3) Running 상태의 프로세스 처리

앞서 2-2-1)과 2-2-2)를 통해 Preemption을 허용하는지, 어떠한 우선순위를 기준으로 하는지에 따라 Ready Queue 관리와 current_process 지정까지 마치게 되었다. 이 과정을 거친뒤 확정된 current_process를 이제 CPU에서 처리해야 한다. current_process를 CPU에서 처리하는 과정에서 context switching이 발생하는 경우는 크게 두 가지이다. 첫 번째로는 프로세스의 remaining_time이 0이 되는 것이다. 즉, 남아있는 Burst Time이 존재하지 않아 프로세스의 작업이 완료되는 것이다. 두 번째로 I/O 요청이 있다. I/O 요청에 도달하게 되면, 그 즉시 context switching이 발생하여 Running 상태에서 Waiting 상태로 전환되게 된다. Round Robin의 경우 예외로 “Time Slice expired”인 상황도 존재하는데 이는 Round Robin 스케줄링 알고리즘의 특수한 경우로 뒤에서 다루도록 하고 지금은 이를 제외한 나머지 5개의 알고리즘에 공통적으로 적용할 수 있는 구조를 다룬다.

```
if(current_process != NULL)
    int elapsed_time = current_process->burst_time - current_process->remaining_time
    if(current_process->io_index < current_process->io_count && elapsed_time+1 ==
      current_process->io_request_time[current_process->io_index])
        current_process->remaining_time--
        current_process의 상태 = waiting 상태
        enqueue(waiting_queue, current_process)
        if(!is_empty(ready_queue))
            current_process = dequeue(ready_queue)
            current_process의 상태 = running 상태
        end(if문) else
            current_proces = NULL
        end(else문)
    end(if문) else
        current_process->remaining_time--
        if(current_process->remaining_time == 0)
            current_process의 상태 = completed 상태
        if(!is_empty(ready_queue))
            current_process = dequeue(ready_queue)
            current_process의 상태 = running 상태
        end(if문) else
            current_process = NULL
        end(else문)
    end(else문)
end(if문)
```

위의 슈도코드는 프로세스의 종료, 프로세스에서의 I/O 요청을 고려한 슈도코드이다. 복잡해 보이지만 그 구조를 살펴보면 단순하다.



2-2-2)에 지정된 Current Process를 먼저 I/O 요청시간에 도달했는지를 확인한다. 만약 도달했을 경우, 우선 remaining_time을 하나 감소시킨다. 그 뒤 Current Process를 Running 상태에서 Waiting 상태로 전환한 뒤, Ready Queue에서 프로세스 하나를 dequeue하게 된다. 만약 Ready 상태가 비어있다면 NULL로 두게 된다. Waiting 상태에서의 처리는 별도의 알고리즘을 통하여 처리하게 된다. 만약 I/O 요청 시간에 도달하지 않았을 경우, 전자와 마찬가지로 remaining_time을 하나 감소시킨다. 그 결과 remaining_time이 0인 경우, 즉 burst time이 남아있지 않은 경우 프로세스가 다 수행되었다는 뜻이다. 이때 현재의 프로세스를 Completed 상태로 바꾼 후 Ready Queue의 상황에 따라 새로 Current Process를 지정하게 된다. 이렇게 새로 지정된 프로세스들은 다시 2-2-2)의 알고리즘들에 의해 preemption 검사를 거친 뒤 순서도의 흐름을 계속 반복하게 된다.

2-2-4) Waiting Queue 관리

지금까지 Ready 상태에서 Running상태, 다시 Running에서 Ready 상태로의 context switching 구조를 구현하였다. 그렇다면 I/O 요청에 의해 프로세스가 Waiting 상태로 전환되었을 때 프로세스는 어떠한 과정을 거칠까? 궁극적으로는 Waiting Queue에서 개별 I/O 요청마다 고유한 burst time인 io_burst_time을 소모한 뒤 다시 Ready Queue에 들어오게 된다. 이를 슈도코드로 표현하면 다음과 같다.

```
1. if(io_process == NULL && !is_empty(waiting_queue))
2.      io_process = dequeue(waiting_queue)
3.      io_remaining_time = io_process의 i번째 io_burst_time
4. end(if문)
5. if(io_process != NULL)
6.      io_remaining_time--
7.      if(io_remaing_time == 0)
8.              io_process의 io_index를 I에서 i+1로 업데이트
9.              io_process->in_ready, in_waiting, in_running = false
10.             io_process->ready_time = time+1
11.             io_process = NULL
12.     end(if문)
13. end(if문)
```

먼저 1~4번의 if문은 2-2-2)와 유사하게 현재 waiting 상태에서 처리할 프로세스를 io_process라는 변수를 인자를 활용하여 할당한다. 이때의 특징이라면은 io_remaining_time을 통해 CPU에서의 remaining_time 역할을 i번째 I/O 내부에서 수행하게 된다. 이후 io_remaing_time을 하나 줄여가며 0에 도달한 경우 그 프로세스에서 현재 처리하고 있는 I/O의 index를 하나 늘려 현재의 I/O는 처리되었음을 공시한다. 그 뒤의 작업은 프로세스를 다시 Ready 상태로 돌려놓기 위한 작업들이다. 9번 라인은 현재의 프로세스를 어느 상태에도 속하지 않게 함으로서 다음 분기에 Ready 상태에 들어가도록 만든다. 이때의 타이밍이 중요한데 time+1을 ready_time으로 지정함으로서 이른 타이밍에 waiting queue에 들어가는 것을 방지한다. 이 모든 과정을 토대로 스케줄링 알고리즘이 구현된다.

3) 스케줄링 알고리즘

3-1) FCFS

FCFS은 First Come, First Service 의 약자로 먼저 Ready Queue에 들어온 순서대로 Running 상태로 전환이 이루어진다. 가장 간단한 구조로 2-2)의 스케줄링 알고리즘의 기본 토대에서 크게 벗어나지 않은 채로 구현이 가능하다.

```
void FCFS(Process, n)
    ready_queue, waiting_queue 생성 및 초기화
    int time, completed, io_remaining_time = 0
    Process current_process, io_process = NULL
    while(completed < n)
        /* Ready Queue 관리(2-2-1) */
        /* Current Process 지정(2-2-2) */
        Ready Queue에 존재하는 프로세스의 waiting_time++
        /* Running 상태의 프로세스 처리(2-2-3) */
        time++
        /* Waiting Queue 관리(2-2-4) */
    end(while문)
end(함수)
```

위는 FCFS 알고리즘의 구조이다. 먼저 알고리즘에 사용될 변수 및 인자들을 초기화한다. 여기서 time과 completed에 대해서 설명을 덧붙일 필요가 있는데 먼저 time은 알고리즘 내에서의 시간을 나타낸다. 이 시간을 기준으로 프로세스의 ready_time, finish_time이 정해지며, 이를 바탕으로 waiting_time, turnaround_time이 계산되어진다. 이 값들은 나중에 알고리즘 간 성능을 비교하는데 사용되어진다. completed의 경우, 완료된 프로세스의 개수를 의미한다. FCFS에서의 인자인 n은 프로세스의 개수를 의미한다. 완료된 프로세스의 개수가 n개라면, 즉, terminated된 프로세스가 전부 다라면 함수를 종료하게 된다. 거꾸로 말한다면 모든 프로세스가 종료될 때까지 <Ready Queue 관리 -> Current Process 지정 -> Running 상태의 프로세스 처리 -> Waiting Queue 관리 -> Ready Queue 관리> 의 시퀀스를 반복하게 된다. Ready Queue 관리, Current Process에서 모두 기본적이 틀을 통해 FCFS를 구현할 수 있기에 가장 구현하는데 간단하다는 점이 특징이다.

3-2) Nonpreemptive SJF

Nonpreemptive SJF 스케줄링 알고리즘은 preeption의 발생을 허용하지 않은 채 가장 빨리 끝나는 작업을 우선시하여 스케줄링하는 알고리즘이다.

```
1. void Nonpreemptive_SJF(Process, n)
2.      ready_queue, waiting_queue 생성 및 초기화
3.      int time, completed, io_remaining_time = 0
4.      Process current_process, io_process = NULL
5.      while(completed < n)
6.              /* Ready Queue 관리(2-2-1) */
7.              sort_ready_queue_by_remaining_time(ready_queue)
8.              /* Current Process 지정(2-2-2) */
9.              Ready Queue에 존재하는 프로세스의 waiting_time++
10.             /* Running 상태의 프로세스 처리(2-2-3) */
11.             time++
12.             /* Waiting Queue 관리(2-2-4) */
13.     end(while문)
14. end(함수)
```

FCFS와 비교했을 때 7번 라인이 추가된 것 이외에 다른 점이 존재하지 않는다. 즉, Ready Queue를 관리하는 단계에서 FCFS와 비교했을 때 remaining_time을 오름차순으로 정렬하는 과정이 추가된 점 이외에 구현되는 원리는 동일하다.

3-3) Nonpreemptive Priority

Shortet Job에서 Priority로 우선순위를 바꾼 것 외에 3-2)의 Nonpreemptive SJF 와 다른 점이 존재하지 않는다. 그렇기에 슈도코드 또한 상당히 유사한데,

```
1. void Nonpreemptive_Priority(Process, n)
2.      ready_queue, waiting_queue 생성 및 초기화
3.      int time, completed, io_remaining_time = 0
4.      Process current_process, io_process = NULL
5.      while(completed < n)
6.              /* Ready Queue 관리(2-2-1) */
7.              sort_ready_queue_by_priority(ready_queue)
8.              /* Current Process 지정(2-2-2) */
9.              Ready Queue에 존재하는 프로세스의 waiting_time++
10.             /* Running 상태의 프로세스 처리(2-2-3) */
11.             time++
12.             /* Waiting Queue 관리(2-2-4) */
13.     end(while문)
14. end(함수)
```

이 또한 3-2)와 비교하면 알 수 있듯 7번에서 적용하는 함수가 sort_ready_queue_by_remaining_time에서 sort_ready_queue_by_priority로 바뀐 것 이외에는 차이점이 존재하지 않는다.

3-4) Preemptive SJF

이번에는 preemption을 허용하는 스케줄링 알고리즘인 Preemptive SJF의 구조에 대해서 살펴보자.

```
1. void Preemptive_SJF(Process, n)
2.      ready_queue, waiting_queue 생성 및 초기화
3.      int time, completed, io_remaining_time = 0
4.      Process current_process, io_process = NULL
5.      while(completed < n)
6.              /* Ready Queue 관리(2-2-1) */
7.              sort_ready_queue_by_remaining_time(ready_queue)
8.              /* Current Process 지정(2-2-2)(Preemption 방식 적용) */
9.              Ready Queue에 존재하는 프로세스의 waiting_time++
10.             /* Running 상태의 프로세스 처리(2-2-3) */
11.             time++
12.             /* Waiting Queue 관리(2-2-4) */
13.     end(while문)
14. end(함수)
```

3-2)의 Nonpreemptive SJF와의 차이점은 8번 라인에 있다. Current Process를 지정할 때 preemption 방법의 알고리즘을 적용한다. 즉, Ready Queue의 front 프로세스와 Current Process의 remaining_time을 비교하는 조건을 하나 더 추가하는 것이다. 이 과정을 통해 선정된 Current Process를 최종적으로 Running 상태에서 작업을 수행하게 된다.

3-5) Preemptive Priority

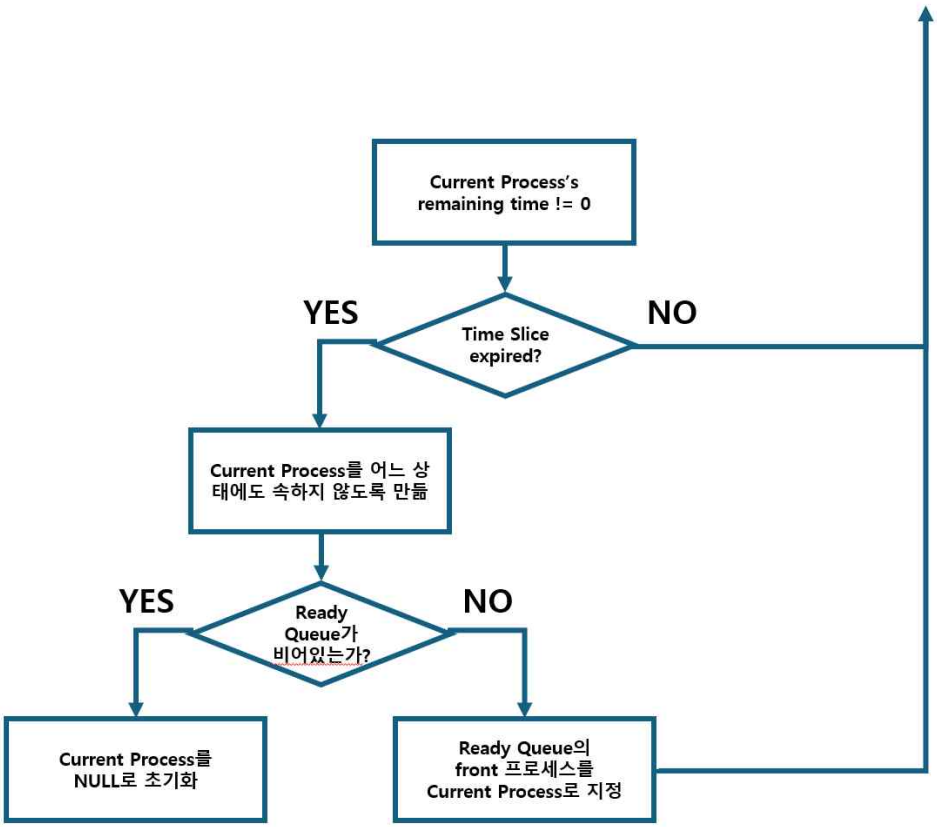
그렇다면 자연스레 Preemptive Priority는 3-3)에서의 Ready 상태를 정렬하는 함수를 추가하고, 3-4)에서의 Current Process를 지정할 때 preemption 방식을 적용한다는 것을 섞은 결과로 나타날 것이다.

```
1. void Preemptive_SJF(Process, n)
2.      ready_queue, waiting_queue 생성 및 초기화
3.      int time, completed, io_remaining_time = 0
4.      Process current_process, io_process = NULL
5.      while(completed < n)
6.              /* Ready Queue 관리(2-2-1) */
7.              sort_ready_queue_by_priority(ready_queue)
8.              /* Current Process 지정(2-2-2)(Preemption 방식 적용) */
9.              Ready Queue에 존재하는 프로세스의 waiting_time++
10.             /* Running 상태의 프로세스 처리(2-2-3) */
11.             time++
12.             /* Waiting Queue 관리(2-2-4) */
13.     end(while문)
14. end(함수)
```

7번 라인에서 sort_ready_queue_by_priority 함수를 통해 Priority를 기준으로 Ready Queue를 정렬하고, 8번 라인에서 preemption 방식을 적용하여 Current Process를 정하게 된다.

3-6) Round Robin

지금까지 알아본 스케줄링 알고리즘과는 살짝 다르게 Round Robin 방식은 time quantum의 개념이 도입된다. Round Robin은 여러 프로세스에게 동일한 시간량을 순차적으로 할당하는 방식인데 여기서 동일한 시간량을 의미하는 말이 time quantum이다. 이는 time slice라고도 불리는데, 앞서서 몇 번 언급했던 time slice expired의 그것이다. 한 작업이 주어진 시간만큼 CPU를 사용하면, 다음 작업으로 순환적으로 교체되어지며, 해당 작업이 끝나지 않았다면, 나중에 다시 돌아와서 할당된 시간만큼 또다시 실행하게 된다. 즉, Running 상태에서 프로세스를 처리하는 과정에서 I/O 요청, 프로세스의 완료에 time slice expired를 추가하여 구현하는 것이다. 2-2-3)의 순서도에서 'Current Process의 remaining time이 0인가?' 의 조건문에서 NO인 경우, 바로 초기의 항목인 '2-2-2)에 의해 Current Process 지정'으로 돌아가는 것이 아니라 'Time slice expired?' 의 분기를 거치게 되는 것이다.



10번 라인의 Running 상태의 프로세스 처리는 2-2-3)의 순서도에서 'Current Prcoess의 remaining time이 0인가?' 의 NO에서 다시 원래의 항목으로 돌아가는 것이 아닌, 바로 위와 순서도가 추가적으로 삽입되는 구조를 지니게 된다.본 프로젝트는 전역변수이 TIME_QUANTUM을 활용하여 초기에 설정해주고, 이를 time_distance라는 지역변수에 차용하여 분기마다 remaining_time를 하나씩 줄이듯이 time_distance를 TIME_QUANTUM에서 하나씩 줄여가며 time slice를 구현하였다. 만약 I/O 요청, 프로세스의 completed, Time slice expired와 같은 변수가 발생하게 되면 현재의 시간 분기를 초기화해주어야 하기에 이를 구현하고자 time_distance를 다시 전역변수인 TIME_QUANTUM으로 초기화하여 문제없이 스케줄링되도록 하였다.

4) 실행결과

3)에서 직접 구현한 6개의 알고리즘을 돌려보면 다음과 같다.

```
Process ID      Arrival Time    Burst Time      I/O Count       Priority
1               1               7               2               3
2               0               2               1               1
3               2               19              0               2
I/O Request Times and Burst Times:
Process 1: (2, 3) (6, 1)
Process 2: (1, 2)
Process 3:
```

위의 프로세스들을 6개의 스케줄링 알고리즘에 차례대로 구현해본다.

```
FCFS

Process 2 started at time 0
Process 2 I/O request at time 1
Process 1 started at time 1
Process 1 I/O request at time 3
Process 3 started at time 3
Process 3 completed at time 22
Process 2 started at time 22
Process 2 completed at time 23
Process 1 started at time 23
Process 1 I/O request at time 27
Process 1 started at time 28
Process 1 completed at time 29
```

```
Nonpreemptive SJF

Process 2 started at time 0
Process 2 I/O request at time 1
Process 1 started at time 1
Process 1 I/O request at time 3
Process 3 started at time 3
Process 3 completed at time 22
Process 2 started at time 22
Process 2 completed at time 23
Process 1 started at time 23
Process 1 I/O request at time 27
Process 1 started at time 28
Process 1 completed at time 29
```

```
Nonpreemptive Priority

Process 2 started at time 0
Process 2 I/O request at time 1
Process 1 started at time 1
Process 1 I/O request at time 3
Process 3 started at time 3
Process 3 completed at time 22
Process 2 started at time 22
Process 2 completed at time 23
Process 1 started at time 23
Process 1 I/O request at time 27
Process 1 started at time 28
Process 1 completed at time 29
```

```
Preemptive SJF

Process 2 started at time 0
Process 2 I/O request at time 1
Process 1 started at time 1
Process 1 I/O request at time 3
Process 3 started at time 3
Process 3 preempted at time 3
Process 2 started at time 3
Process 2 completed at time 4
Process 3 started at time 4
Process 3 preempted at time 6
Process 1 started at time 6
Process 1 I/O request at time 10
Process 3 started at time 10
Process 3 preempted at time 11
Process 1 started at time 11
Process 1 completed at time 12
Process 3 started at time 12
Process 3 completed at time 28
```

```
Preemptive Priority

Process 2 started at time 0
Process 2 I/O request at time 1
Process 1 started at time 1
Process 1 preempted at time 2
Process 3 started at time 2
Process 3 preempted at time 3
Process 2 started at time 3
Process 2 completed at time 4
Process 1 started at time 4
Process 1 preempted at time 4
Process 3 started at time 4
Process 3 completed at time 22
Process 1 started at time 22
Process 1 I/O request at time 23
Process 1 started at time 26
Process 1 I/O request at time 30
Process 1 started at time 31
Process 1 completed at time 32
```

```
Round Robin (RR)

Process 2 started at time 0
Process 2 I/O request at time 1
Process 1 started at time 1
Process 1 I/O request at time 3
Process 3 started at time 3
Process 3 preempted at time 8
Process 2 started at time 8
Process 2 completed at time 9
Process 1 started at time 9
Process 1 I/O request at time 13
Process 3 started at time 13
Process 3 preempted at time 18
Process 1 started at time 18
Process 1 completed at time 19
Process 3 started at time 19
Process 3 preempted at time 24
Process 3 started at time 24
Process 3 completed at time 28
```

단순히 스케줄링 알고리즘을 작동시키면 잘 스케줄링한 것인지 확인이 어렵기에 context switching이 발생할 때마다 현재의 상황을 나타내는 문구를 출력하도록 구현한 결과이다. 6개의 알고리즘 모두 정상적으로 잘 작동하는 것을 알 수 있다.

5) 알고리즘 간 성능 비교

```
FCFS Evaluation:
Process ID      Waiting Time    Turnaround Time
1               17              28
2               19              23
3               1               20
Average Waiting Time: 12.33
Average Turnaround Time: 23.67
```

```
Nonpreemptive SJF Evaluation:
Process ID      Waiting Time    Turnaround Time
1               17              28
2               19              23
3               1               20
Average Waiting Time: 12.33
Average Turnaround Time: 23.67
```

```
Nonpreemptive Priority Evaluation:
Process ID      Waiting Time    Turnaround Time
1               17              28
2               19              23
3               1               20
Average Waiting Time: 12.33
Average Turnaround Time: 23.67
```

```
Preemptive SJF Evaluation:
Process ID      Waiting Time    Turnaround Time
1               2               11
2               1               4
3               6               26
Average Waiting Time: 3.00
Average Turnaround Time: 13.67
```

```
Preemptive Priority Evaluation:
Process ID      Waiting Time    Turnaround Time
1               18              31
2               0               4
3               0               20
Average Waiting Time: 6.00
Average Turnaround Time: 18.33
```

```
Round Robin Evaluation:
Process ID      Waiting Time    Turnaround Time
1               7               18
2               5               9
3               7               26
Average Waiting Time: 6.33
Average Turnaround Time: 17.67
```

위의 이미지들은 4)에서의 스케줄링 결과를 Waiting Time과 Turnaround Time을 통하여 성능을 비교한 것이다. Average Waiting Time, Average Turnarount Time 두 지표 모두 Preemptive SJF 방법이 4)에서의 프로세스들을 스케줄링하는 데 가장 적합한 방법이라고 말할 수 있다.

결론

이번 프로젝트에서는 다양한 CPU 스케줄링 알고리즘(FCFS, Nonpreemptive SJF, Nonpreemptive Priority, Preemptive SJF, Preemptive Priority, Round Robin)을 직접 구현한 시뮬레이터를 개발하였다. 본 시뮬레이터는 기존 오픈소스 시뮬레이터들과 달리, I/O 요청이 포함된 프로세스의 상태 전이와 Context Switching을 실제 운영체제의 동작과 유사하게 처리한다는 점에서 차별성을 가진다. 프로세스는 수동 입력 또는 자동 생성이 가능하며, 각 프로세스는 arrival time, burst time, priority, I/O 요청 정보 등 다양한 속성을 갖는다. 각 알고리즘은 Ready Queue와 Waiting Queue를 효율적으로 관리하고, preemption 및 time slice와 같은 상황을 세밀하게 처리하여 실제 운영체제 환경을 모사하였다. 또한, 시뮬레이션 결과를 바탕으로 Waiting Time과 Turnaround Time을 산출하여 알고리즘 간 성능을 비교할 수 있도록 하였다.

프로젝트를 수행하면서 운영체제의 핵심 개념인 CPU 스케줄링의 원리와 실제 구현상의 어려움을 깊이 있게 체험할 수 있었다. 특히, I/O 요청이 포함된 프로세스의 상태 전이와 Context Switching, 그리고 preemption 처리 로직을 직접 설계하고 구현하면서, 이론과 실제의 차이를 체감할 수 있었다. 여러 알고리즘을 직접 비교해보니, 프로세스 특성에 따라 최적의 스케줄링 방식이 달라질 수 있다는 점도 확인할 수 있었다.

향후 발전 방향에 대해서는, 지금까지 구현한 시뮬레이터가 실제 운영체제의 흐름과 유사하게 다양한 스케줄링 알고리즘을 재현했다는 점에서 의미가 크다고 생각한다. 앞으로는 시뮬레이터를 좀 더 현실적인 환경으로 확장해보고 싶다. 예를 들어, 현재는 단일 CPU와 I/O 장치만을 다루고 있지만, 멀티코어 환경이나 여러 입출력 장치를 지원하도록 구조를 확장하면 실제 시스템과 더 가까워질 것 같다. 이런 방향으로 시뮬레이터를 계속 발전시켜 나간다면, 운영체제의 복잡한 동작을 더 깊이 이해하고 체험할 수 있는 좋은 도구가 될 거라고 생각한다.